

Guía de producción — Vuelvo Comercios

App Android (Jetpack Compose) · `com.delta.vuelvo_commerce` · Generada 2026-06-16 · Lanzamiento: commerce primero, luego client

Estado real del repo hoy (lo que parece hecho pero son placeholders)

- **Billing 7.1.1** está en el catálogo pero la suscripción es un `boolean` in-memory en `BizApp.kt` → **no hay flujo de compra real**.
- **Firestore desactivado**: plugins/deps comentados y `app/google-services.json` es un **placeholder falso** (`project_number: 000000`, api key falsa).
- Release build con `isMinifyEnabled = false` y **sin** `signingConfig` (solo firma debug). **No existe keystore**.
- Datos in-memory y NFC solo animación → **sin backend ni persistencia**.

La UI está; **toda la capa de producción está por hacer**. Esa es la ruta.

FASE 0

Cuentas y decisiones de negocio

1. **Google Play Console** — registro de desarrollador, pago único de **25 USD**.
 - **Organización**: requiere número **D-U-N-S**; sin requisito de testing previo. Recomendado para app comercial.
 - **Personal**: más rápida, pero cuentas nuevas deben correr **closed testing con ~12 testers durante 14 días** antes de producción.
 - Verificación de identidad obligatoria — empieza ya, tarda.
2. **Perfil de pagos de Google (merchant)** — obligatorio para **vender suscripciones**. Necesita datos fiscales/bancarios.
3. **Política de privacidad publicada (URL)** — obligatoria para la ficha y el Data Safety form.

España — ¿hay que ser autónomo? Para la cuenta **Organización** necesitas un **D-U-N-S**, que se da a una entidad con actividad registrada; en España es viable **siendo autónomo** (no hace falta S.L.). Pero lo de fondo es Hacienda: para **cobrar suscripciones de forma recurrente** debes estar dado de alta (**autónomo**: modelo 036/037 + RETA, o una sociedad). La cuenta Personal te ahorra el D-U-N-S, no el alta fiscal. Vía habitual: **alta autónomo** → **D-U-N-S** → **cuenta Organización**. Confírmalo con una gestoría.

FASE 1

Identidad y firma de la app **BLOQUEANTE**

4. Confirmar el `applicationId` **definitivo** — tras publicar **no se puede cambiar nunca**. `com.delta.vuelvo_commerce` sirve; solo confírmalo.
5. **Generar el upload keystore** (`.jks`) y guardar archivo + contraseñas a buen recaudo (perderlo = no poder actualizar la app).
6. Activar **Play App Signing** (Google custodia la clave final; tú subes con tu upload key).

7. En `app/build.gradle.kts` : añadir `signingConfig` de release (credenciales en `keystore.properties` fuera de git), poner `isMinifyEnabled = true` + R8/proguard, y definir estrategia de `versionCode` .

FASE 2

Firestore real

8. **Decisión de arquitectura:** Comercios y Client deberían compartir **UN proyecto Firebase** (mismo Firestore/Auth) con **dos apps Android registradas**. Decídelo antes de esta fase.
9. Crear el proyecto real → registrar `com.delta.vuelvo_commerce` con los **SHA-1/SHA-256** (del upload key **y** de la clave de Play App Signing).
10. Descargar el `google-services.json` **real** y reemplazar el placeholder.
11. El `libs.versions.toml` ya está listo; descomentar plugins y deps. Empezar con **Auth, Firestore, Crashlytics, Analytics**. Messaging/Functions en Fase 4.

Compartido vs separado — trade-offs

Opción	A favor	En contra
Compartido (recomendado)	Un Firestore: el sello del comercio lo lee el cliente sin sincronizar nada (flujo central). Un solo Auth/Functions/Pub-Sub. Más barato y simple. Dos <code>google-services.json</code> es normal.	Menos aislamiento entre datos de ambas apps.
Separado	Aislamiento fuerte; cuotas/facturación independientes.	Hay que construir una capa de sincronización entre backends — y eso es justo el corazón de la app. Más latencia y consistencia.

FASE 3

Backend y persistencia **BLOQUEANTE**

12. La app es in-memory: necesitas **Auth** (alta/login de comercios) + **Firestore** (modelo: comercios, tags NFC, estado de suscripción). Sin esto es un demo, no un producto.
13. Reglas de seguridad de Firestore (cada comercio solo ve lo suyo).

FASE 4

Suscripción — Play Billing real **BLOQUEANTE**

14. En Play Console → **Productos de suscripción**: crear suscripción con sus **base plans** y **offers** (mensual/anual, prueba gratis). Los IDs deben coincidir con el código.
15. Integrar **BillingClient** de verdad: conexión → `queryProductDetails` → `launchBillingFlow` → escuchar `PurchasesUpdated` → `acknowledge` . Reemplaza el `boolean` de `BizApp.kt` .
16. **Validación en servidor** (no confíes en el cliente): Cloud Function con la **Play Developer API** + **Real-time Developer Notifications (RTDN)** vía Pub/Sub para renovaciones/cancelaciones. El **entitlement** se guarda en Firestore.

17. Verifica la **versión mínima de Billing Library** exigida en 2026 (suele forzar la última mayor); quizá haya que subir de 7.1.1.

FASE 5

Cumplimiento y ficha de Play

18. **Data safety form**, **content rating**, **target audience**, declaración de **app de pagos/suscripciones** y justificación del permiso **NFC**.

19. Ficha: icono 512×512, feature graphic, screenshots, descripción corta/larga, URL de privacidad.

FASE 6

Testing tracks

20. **Internal testing** (hasta 100 testers) → probar compras con **license testers** (sandbox, sin cobro) → **Closed** → **Producción**.

21. Si la cuenta es Personal: aquí se cumplen los 14 días / 12 testers.

FASE 7

Release

22. Generar **AAB** firmado ([bundleRelease](#)), subir, **staged rollout** (ej. 20%) y monitorizar Crashlytics.

Resumen de bloqueantes

Sin esto no se publica nada funcional: **Fase 1** (firma), **Fase 2** (Firebase real), **Fase 3** (backend) y **Fase 4** (billing + validación servidor). Las fases 0 y 5–7 son trámite de Play.

Recomendación de arranque: empezar por **Fase 1 (firma de release)** — no depende de ninguna decisión pendiente — y en paralelo tramitar el **alta de autónomo + D-U-N-S** (los tiempos muertos largos).